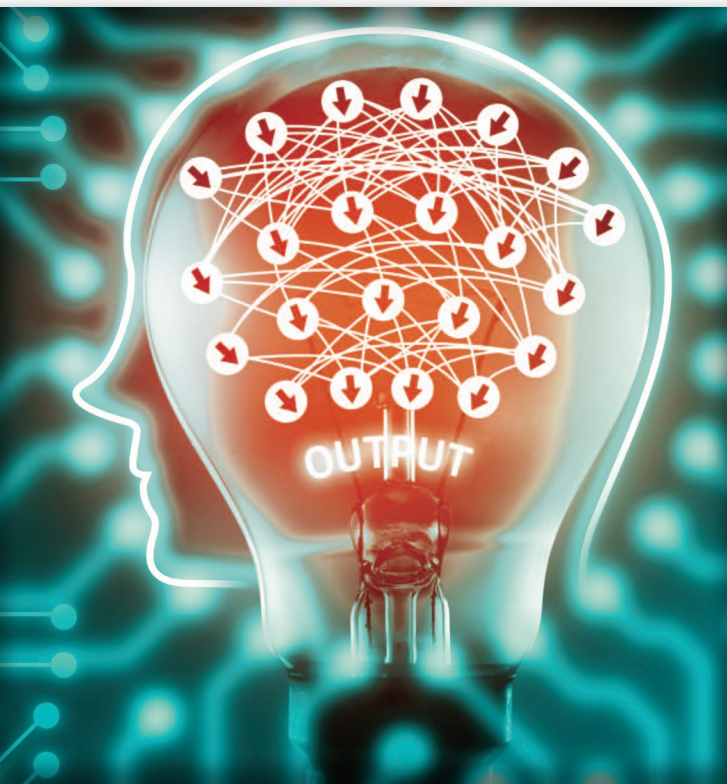


Antonia Creswell, Tom White, Vincent Dumoulin,
Kai Arulkumaran, Biswa Sengupta, and Anil A. Bharath

Generative Adversarial Networks

An overview



©ISTOCKPHOTO.COM/ZAPP2PHOTO

Generative adversarial networks (GANs) provide a way to learn deep representations without extensively annotated training data. They achieve this by deriving backpropagation signals through a competitive process involving a pair of networks. The representations that can be learned by GANs may be used in a variety of applications, including image synthesis, semantic image editing, style transfer, image superresolution, and classification. The aim of this review article is to provide an overview of GANs for the signal processing community, drawing on familiar analogies and concepts where possible. In addition to identifying different methods for training and constructing GANs, we also point to remaining challenges in their theory and application.

Introduction

GANs are an emerging technique for both semisupervised and unsupervised learning. They achieve this through implicitly modeling high-dimensional distributions of data. Proposed in 2014 [1], they can be characterized by training a pair of networks in competition with each other. A common analogy, apt for visual data, is to think of one network as an art forger and the other as an art expert. The forger, known in the GAN literature as the *generator*, $\mathcal{G}$, creates forgeries, with the aim of making realistic images. The expert, known as the *discriminator*, $\mathcal{D}$, receives both forgeries and real (authentic) images, and aims to tell them apart (see Figure 1). Both are trained simultaneously, and in competition with each other.

Crucially, the generator has no direct access to real images—the only way it learns is through its interaction with the discriminator. The discriminator has access to both the synthetic samples and samples drawn from the stack of real images. The error signal to the discriminator is provided through the simple ground truth of knowing whether the image came from the real stack or from the generator. The same error signal, via the discriminator, can be used to train the generator, leading it toward being able to produce forgeries of better quality.

The networks that represent the generator and discriminator are typically implemented by multilayer networks consisting

Digital Object Identifier 10.1109/MSP.2017.2765202
Date of publication: 9 January 2018

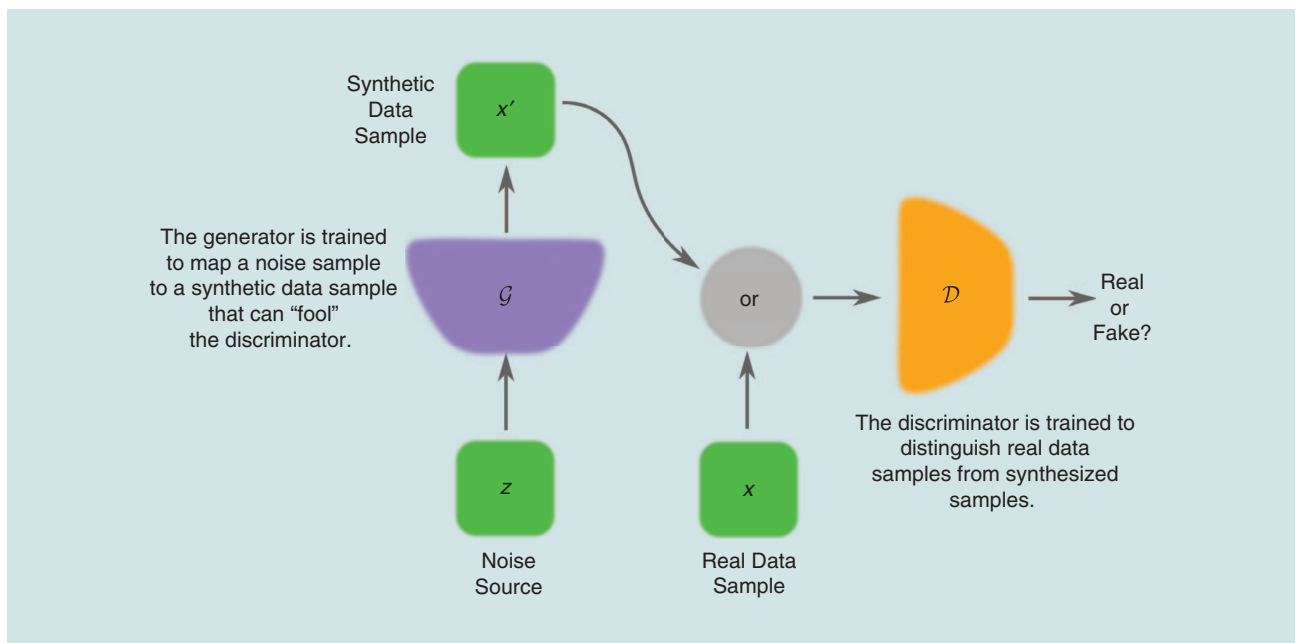


FIGURE 1. The two models that are learned during the training process for a GAN are the discriminator ($\mathcal{D}$) and the generator ($\mathcal{G}$). These are typically implemented with neural networks, but they could be implemented by any form of differentiable system that maps data from one space to another; see article text for details.

of convolutional and/or fully connected layers. The generator and discriminator networks must be differentiable, though it is not necessary for them to be directly invertible. If one considers the generator network as mapping from some representation space, called a *latent space*, to the space of the data (we shall focus on images), then we may express this more formally as $\mathcal{G}: \mathcal{G}(\mathbf{z}) \rightarrow \mathcal{R}^{|\mathbf{x}|}$, where $\mathbf{z} \in \mathcal{R}^{|\mathbf{z}|}$ is a sample from the latent space, $\mathbf{x} \in \mathcal{R}^{|\mathbf{x}|}$ is an image and $|\cdot|$ denotes the number of dimensions.

In a basic GAN, the discriminator network, $\mathcal{D}$, may be similarly characterized as a function that maps from image data to a probability that the image is from the real data distribution, rather than the generator distribution: $\mathcal{D}: \mathcal{D}(\mathbf{x}) \rightarrow (0, 1)$. For a fixed generator, $\mathcal{G}$, the discriminator, $\mathcal{D}$, may be trained to classify images as either being from the training data (real, close to one) or from a fixed generator (fake, close to zero). When the discriminator is optimal, it may be frozen, and the generator, $\mathcal{G}$, may continue to be trained so as to lower the accuracy of the discriminator. If the generator distribution is able to match the real data distribution perfectly, then the discriminator will be maximally confused, predicting 0.5 for all inputs. In practice, the discriminator might not be trained until it is optimal; we explore the training process in more depth in the section "Training GANs."

On top of the interesting academic problems related to training and constructing GANs, the motivations behind training GANs may not necessarily be the generator or the discriminator per se: the representations embodied by either of the pair of networks can be used in a variety of subsequent tasks. We explore the applications of these representations in the section "Application of GANs."

Preliminaries

Terminology

Generative models learn to capture the statistical distribution of training data, allowing us to synthesize samples from the learned distribution. On top of synthesizing novel data samples, which may be used for downstream tasks such as semantic image editing [2], data augmentation [3], and style transfer [4], we are also interested in using the representations that such models learn for tasks such as classification [5] and image retrieval [6].

We occasionally refer to fully connected and convolutional layers of deep networks; these are generalizations of perceptrons or spatial filter banks with nonlinear postprocessing. In all cases, the network weights are learned through backpropagation [7].

Notation

The GAN literature generally deals with multidimensional vectors and often represents vectors in a probability space by italics (e.g., latent space is $\mathbf{z}$). In the field of signal processing, it is common to represent vectors by bold, lowercase symbols, and we adopt this convention to emphasize the multidimensional nature of variables. Accordingly, we will commonly refer to $p_{\text{data}}(\mathbf{x})$ as representing the probability density function over a random vector $\mathbf{x}$ that lies in $\mathcal{R}^{|\mathbf{x}|}$. We will use $p_g(\mathbf{x})$ to denote the distribution of the vectors produced by the generator network of the GAN. We use the calligraphic symbols $\mathcal{G}$ and $\mathcal{D}$ to denote the generator and discriminator networks, respectively. Both networks have sets of parameters (weights), Θ_D and Θ_G , that are learned through optimization, during training.

As with all deep-learning systems, training requires that we have some clear objective function. Following the usual notation, we use $J_G(\Theta_G; \Theta_D)$ and $J_D(\Theta_D; \Theta_G)$ to refer to the objective functions of the generator and discriminator, respectively. The choice of notation reminds us that the two objective functions are, in a sense, codependent on the evolving parameter sets Θ_G and Θ_D of the networks as they are iteratively updated. We shall explore this further in the section “Training GANs.” Finally, note that multidimensional gradients are used in the updates; we use ∇_{Θ_G} to denote the gradient operator with respect to the weights of the generator parameters and ∇_{Θ_D} to denote the gradient operator with respect to the weights of the discriminator. The expected gradients are indicated by the notation $\mathbb{E}\nabla$.

Capturing data distributions

A central problem of signal processing and statistics is that of density estimation: obtaining a representation—implicit or explicit, parametric or nonparametric—of data in the real world. This is the key motivation behind GANs. In the GAN literature, the term *data generating distribution* is often used to refer to the underlying probability density or probability mass function of observation data. GANs learn through implicitly computing some sort of similarity between the distribution of a candidate model and the distribution corresponding to real data (see Figure 2).

Why bother with density estimation at all? The answer lies at the heart of—arguably—many problems of visual inference, including image categorization, visual object detection and recognition, object tracking, and object registration. In principle, through Bayes’ theorem, all inference problems of computer vision can be addressed through estimating conditional density functions, possibly indirectly in the form of a model that learns the joint distribution of variables of interest and the observed data. The difficulty we face is that likelihood functions for high-dimensional, real-world image data are difficult to construct. While GANs don’t explicitly provide a way of evaluating density functions, for a generator-discriminator

pair of suitable capacity, the generator implicitly captures the distribution of the data.

Related work

One may view the principles of generative models by making comparisons with standard techniques in signal processing and data analysis. For example, signal processing makes wide use of the idea of representing a signal as the weighted combination of basis functions. Fixed basis functions underlie standard techniques such as Fourier-based and wavelet representations. Data-driven approaches to constructing basis functions can be traced back to the Hotelling [8] transform, rooted in Pearson’s observation that principal components minimize a reconstruction error according to a minimum squared error criterion. Despite its wide use, standard principal component analysis (PCA) does not have an overt statistical model for the observed data, though it has been shown that the bases of PCA may be derived as a maximum likelihood parameter estimation problem.

Despite wide adoption, PCA is limited—the basis functions emerge as the eigenvectors of the covariance matrix over observations of the input data, and the mapping from the representation space back to signal or image space is linear. So, we have both a shallow and a linear mapping, limiting the complexity of the model and, hence, of the data, that can be represented.

Independent component analysis (ICA) provides another level up in sophistication, in which the signal components no longer need to be orthogonal; the mixing coefficients used to blend components together to construct examples of data are merely considered to be statistically independent. ICA has various formulations that differ in their objective functions used during estimating signal components or in the generative model that expresses how signals or images are generated from those components. A recent innovation explored through ICA is noise contrastive estimation (NCE); this may be seen as approaching the spirit of GANs [9]: the objective function for learning independent components compares a statistic applied

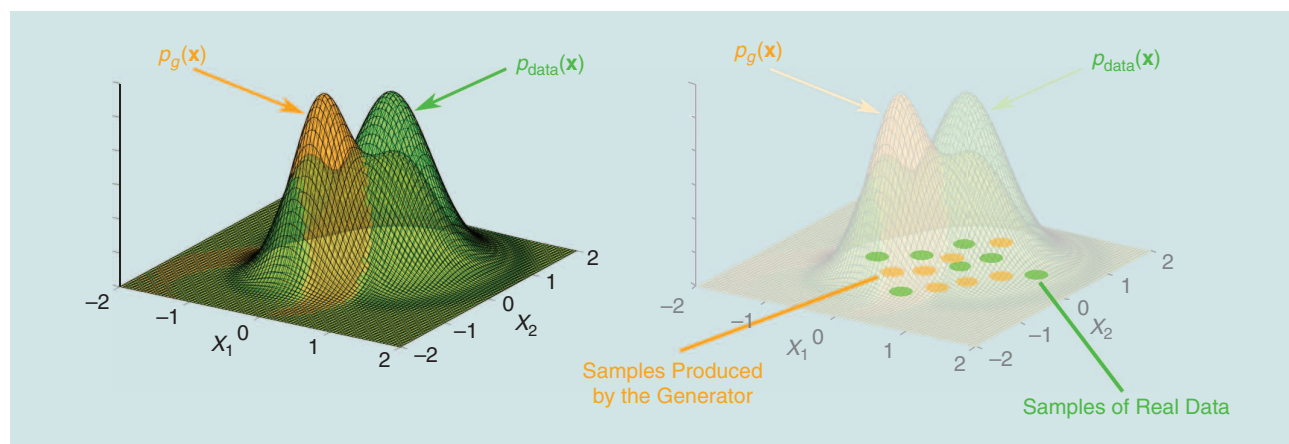


FIGURE 2. During GAN training, the generator is encouraged to produce a distribution of samples, $p_g(\mathbf{x})$ to match that of real data, $p_{data}(\mathbf{x})$. For an appropriately parameterized and trained GAN, these distributions will be nearly identical. The representations embodied by GANs are captured in the learned parameters (weights) of the generator and discriminator networks.

to noise with that produced by a candidate generative model [10]. The original NCE approach did not include updates to the generator.

What other comparisons can be made between GANs and the standard tools of signal processing? For PCA, ICA, Fourier, and wavelet representations, the latent space of GANs is, by analogy, the coefficient space of what we commonly refer to as *transform space*. What sets GANs apart from these standard tools of signal processing is the level of complexity of the models that map vectors from latent space to image space. Because the generator networks contain nonlinearities, and can be of almost arbitrary depth, this mapping—as with many other deep-learning approaches—can be extraordinarily complex.

With regard to deep image-based models, modern approaches to generative image modeling can be grouped into explicit and implicit density models. Explicit density models are either tractable (change of variables models, autoregressive models) or intractable (directed models trained with variational inference, undirected models trained using Markov chains). Implicit

density models capture the statistical distribution of the data through a generative process that makes use of either ancestral sampling [11] or Markov chain-based sampling. GANs fall into the directed implicit model category. A more detailed overview and relevant papers can be found in [12].

GAN architectures

Fully connected GANs

The first GAN architectures used fully connected neural networks for both the generator and discriminator [1]. This type of architecture was applied to relatively simple image data sets: MNIST (handwritten digits), CIFAR-10 (natural images), and the Toronto Face Data Set (TFD).

Convolutional GANs

Going from fully connected to convolutional neural networks (CNNs) is a natural extension, given that CNNs are extremely well suited to image data. Early experiments conducted on

CIFAR-10 suggested that it was more difficult to train generator and discriminator networks using CNNs with the same level of capacity and representational power as those used for supervised learning.

The Laplacian pyramid of adversarial networks (LAPGAN) [13] offered one solution to this problem, by decomposing the generation process using multiple scales: a ground-truth image is itself decomposed into a Laplacian pyramid and a conditional, convolutional GAN is trained to produce each layer given the one above.

Additionally, Radford et al. [5] proposed a family of network architectures called *deep convolutional GAN* (DCGAN), which allows training a pair of deep convolutional generator and discriminator networks. DCGANs make use of strided and fractionally strided convolutions, which allow the spatial downsampling and upsampling operators to be learned during training. These operators handle the change in sampling rates and locations, a key requirement in mapping from image space to possibly lower-dimensional latent space, and from image space to a discriminator. Further details of the DCGAN architecture and training are presented in the section “Training Tricks.”

As an extension to synthesizing images in two dimensions, Wu et al. [14] presented GANs that were able to

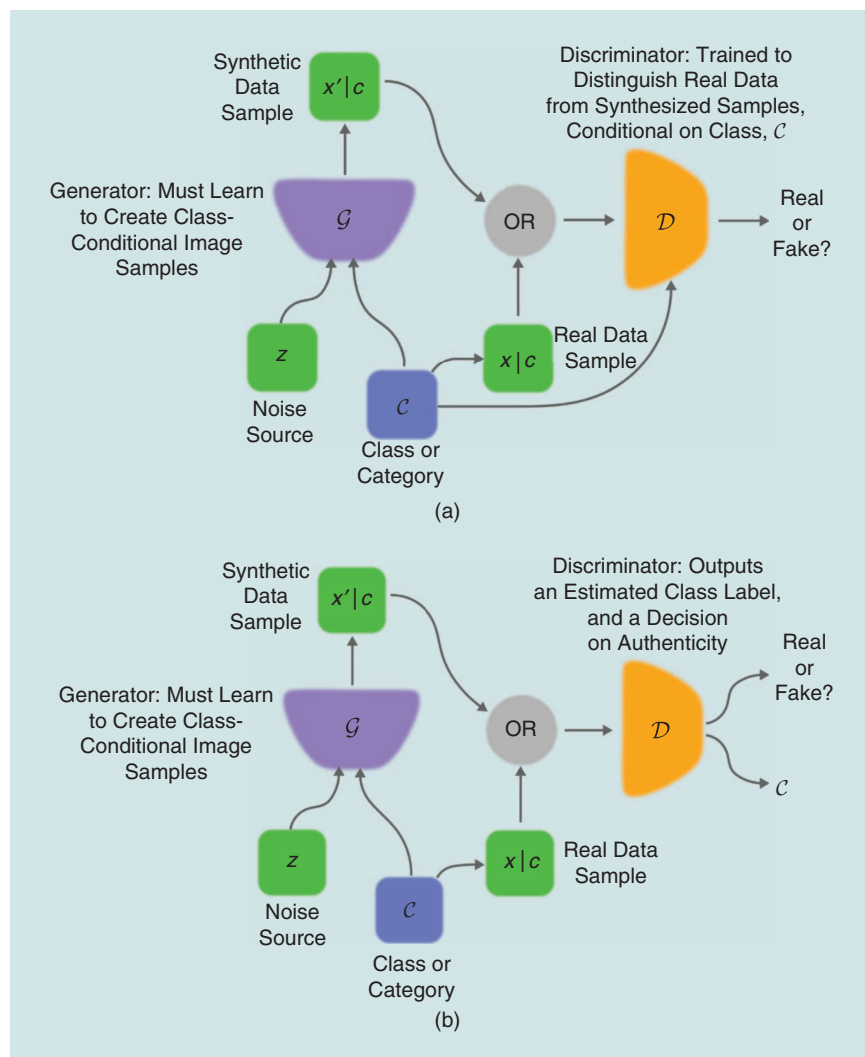


FIGURE 3. (a) The conditional GAN, proposed by Mirza et al. [15] performs class-conditional image synthesis; the discriminator performs class-conditional discrimination of real from fake images. (b) The InfoGAN [16], on the other hand, has a discriminator network that also estimates the class label.

synthesize three-dimensional (3-D) data samples using volumetric convolutions. Wu et al. [14] synthesized novel objects including chairs, a table, and cars; in addition, they also presented a method to map from two-dimensional (2-D) images to 3-D versions of objects portrayed in those images.

Conditional GANs

Mirza et al. [15] extended the (2-D) GAN framework to the conditional setting by making both the generator and the discriminator networks class-conditional (Figure 3). Conditional GANs have the advantage of being able to provide better representations for multimodal data generation. A parallel can be drawn between conditional GANs and InfoGAN [16], which decomposes the noise source into an incompressible source and a “latent code,” attempting to discover latent factors of variation by maximizing the mutual information between the latent code and the generator’s output. This latent code can be used to discover object classes in a purely unsupervised fashion, although it is not strictly necessary that the latent code be categorical. The representations learned by InfoGAN appear to be semantically meaningful, dealing with complex intertangled factors in image appearance, including variations in pose, lighting, and emotional content of facial images [16].

GANs with inference models

In their original formulation, GANs lacked a way to map a given observation, $\mathbf{x}$, to a vector in latent space—in the GAN literature, this is often referred to as an *inference mechanism*. Several techniques have been proposed to invert the generator of pretrained GANs [17], [18]. The independently proposed

adversarially learned inference (ALI) [19] and bidirectional GANs (BiGANs) [20] provide simple but effective extensions, introducing an inference network in which the discriminators examine joint (data, latent) pairs.

In this formulation, the generator consists of two networks: the “encoder” (inference network) and the “decoder.” They are jointly trained to fool the discriminator. The discriminator itself receives pairs of $(\mathbf{x}, \mathbf{z})$ vectors (see Figure 4), and has to determine which pair constitutes a genuine tuple consisting of real image sample and its encoding, or a fake image sample and the corresponding latent-space input to the generator.

Ideally, in an encoding-decoding model, the output, referred to as a *reconstruction*, should be similar to the input. Typically, the fidelity of reconstructed data samples synthesized using an ALI/BiGAN are poor. The fidelity of samples may be improved with an additional adversarial cost on the distribution of data samples and their reconstructions [21].

Adversarial autoencoders

Autoencoders are networks, composed of an encoder and decoder, which learn to map data to an internal latent representation and out again. That is, they learn a deterministic mapping (via the encoder) from a data space, e.g., images, into a latent or representation space, and a mapping (via the decoder) from the latent space back to data space. The composition of these two mappings results in a reconstruction, and the two mappings are trained such that a reconstructed image is as close as possible to the original.

Autoencoders are reminiscent of the perfect-reconstruction filter banks that are widely used in image and signal processing.

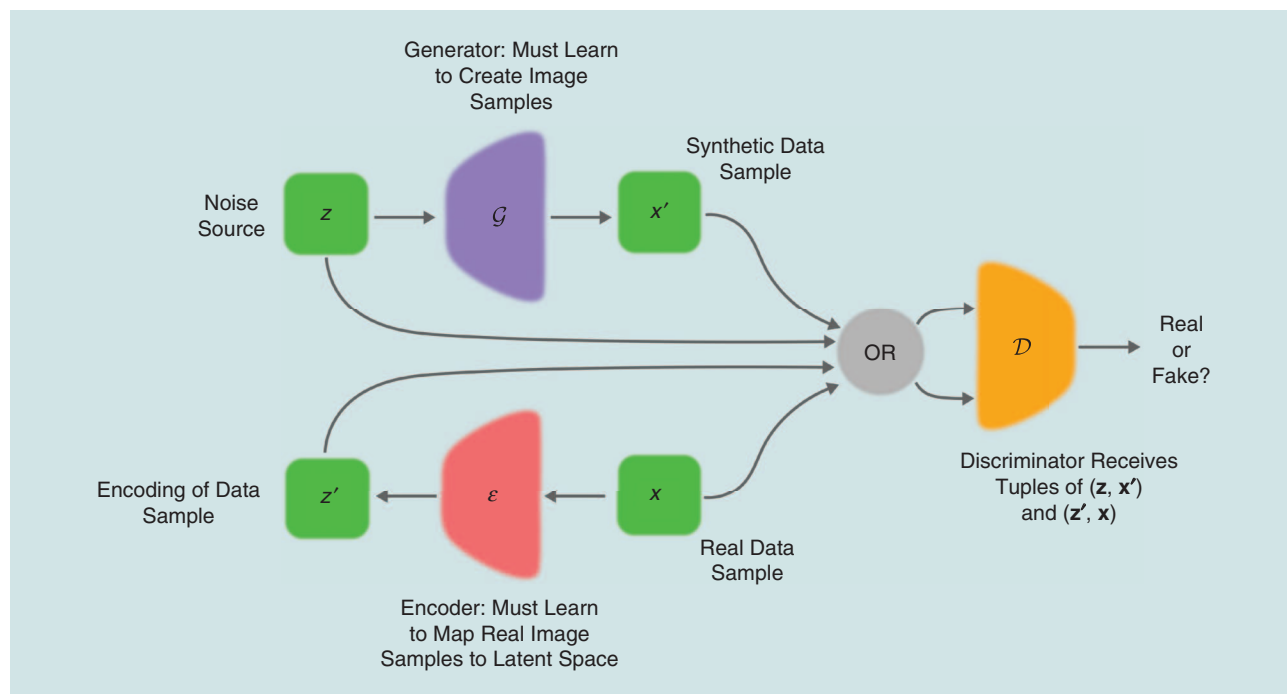


FIGURE 4. The ALI/BiGAN structure [19], [20] consists of three networks. One of these serves as a discriminator, another maps the noise vectors from latent space to image space (decoder, depicted as a generator $\mathcal{G}$ in the figure), with the final network (encoder, depicted as $\mathcal{E}$) mapping from image space to latent space.

However, autoencoders generally learn nonlinear mappings in both directions. Further, when implemented with deep networks, the possible architectures that can be used to implement autoencoders are remarkably flexible. Training can be unsupervised, with backpropagation being applied between the reconstructed image and the original to learn the parameters of both the encoder and the decoder.

As suggested previously, one often wants the latent space to have a useful organization. Additionally, one may want to perform feed-forward, ancestral sampling [11] from an auto-encoder. Adversarial training provides a route to achieve these two goals. Specifically, adversarial training may be applied between the latent space and a desired prior distribution on the latent space (latent-space GAN). This results in a combined loss function [22] that reflects both the reconstruction error and a measure of how different the distribution of the prior is from that produced by a candidate encoding network. This approach is akin to a variational autoencoder (VAE) [23] for which the latent-space GAN plays the role of the Kullback–Leibler (KL)-divergence term of the loss function.

Mescheder et al. [24] unified VAEs with adversarial training in the form of the adversarial variational Bayes (AVB) framework. Similar ideas were presented in [12]. AVB tries to optimize the same criterion as that of VAEs, but uses an adversarial training objective rather than the KL divergence.

Training GANs

Introduction

The training of GANs involves both finding the parameters of a discriminator that maximize its classification accuracy and finding the parameters of a generator that maximally con-

fuse the discriminator. This training process is summarized in Figure 5.

The cost of training is evaluated using a value function, $V(\mathcal{G}, \mathcal{D})$ that depends on both the generator and the discriminator. The training involves solving

$$\max_{\mathcal{D}} \min_{\mathcal{G}} V(\mathcal{G}, \mathcal{D}),$$

where

$$V(\mathcal{G}, \mathcal{D}) = \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \log \mathcal{D}(\mathbf{x}) + \mathbb{E}_{p_{\mathcal{G}}(\mathbf{x})} \log (1 - \mathcal{D}(\mathbf{x})).$$

During training, the parameters of one model are updated, while the parameters of the other are fixed. Goodfellow et al. [1] show that, for a fixed generator, there is a unique optimal discriminator, $\mathcal{D}^*(\mathbf{x}) = p_{\text{data}}(\mathbf{x}) / (p_{\text{data}}(\mathbf{x}) + p_{\mathcal{G}}(\mathbf{x}))$. They also show that the generator, $\mathcal{G}$, is optimal when $p_{\mathcal{G}}(\mathbf{x}) = p_{\text{data}}(\mathbf{x})$, which is equivalent to the optimal discriminator predicting 0.5 for all samples drawn from $\mathbf{x}$. In other words, the generator is optimal when the discriminator, $\mathcal{D}$, is maximally confused and cannot distinguish real samples from ones that are fake.

Ideally, the discriminator is trained until optimal with respect to the current generator; then the generator is again updated. However in practice, the discriminator might not be trained until optimal but rather may only be trained for a small number of iterations, and the generator is updated simultaneously with the discriminator. Further, an alternate, nonsaturating training criterion is typically used for the generator, using $\max_{\mathcal{G}} \log \mathcal{D}(\mathcal{G}(\mathbf{z}))$ rather than $\min_{\mathcal{G}} \log (1 - \mathcal{D}(\mathcal{G}(\mathbf{z})))$.

Despite the theoretical existence of unique solutions, GAN training is challenging and often unstable for several reasons [5], [25], [26]. One approach to improving GAN training is to

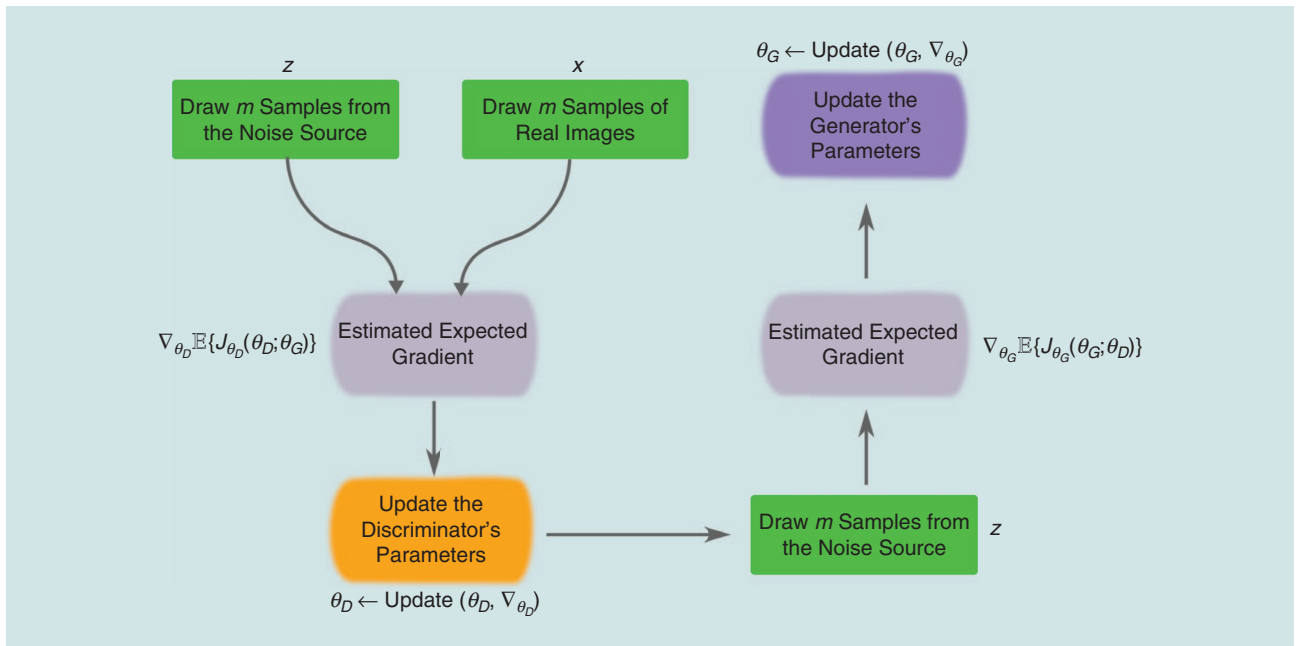


FIGURE 5. The main loop of GAN training. Novel data samples, $\mathbf{x}'$, may be drawn by passing random samples, $\mathbf{z}$, through the generator network. The gradient of the discriminator may be updated k times before updating the generator.

asses the empirical “symptoms” that might be experienced during training. These symptoms include:

- difficulties in getting the pair of models to converge [5]
 - the generative model “collapsing” to generate very similar samples for different inputs [25]
 - the discriminator loss converging quickly to zero [26], providing no reliable path for gradient updates to the generator.
- Several authors suggested heuristic approaches to address these issues [1], [25]; these are discussed in the next section.

Early attempts to explain why GAN training is unstable were proposed by Goodfellow and Salimans et al. [1], [25], who observed that gradient descent methods typically used for updating both the parameters of the generator and discriminator are inappropriate when the solution to the optimization problem posed by GAN training actually constitutes a saddle point. Salimans et al. provided a simple example that shows this [25]. However, stochastic gradient descent is often used to update neural networks and there are well-developed machine-learning programming environments that make it easy to construct and update networks using stochastic gradient descent.

Although an early theoretical treatment [1] showed that the generator is optimal when $p_g(\mathbf{x}) = p_{\text{data}}(\mathbf{x})$, a very neat result with a strong underlying intuition, the real data samples reside on a manifold that sits in a high-dimensional space of possible representations. For instance, if color image samples are of size $N \times N \times 3$ with pixel values $[0, \mathbb{R}^+)^3$, the space that may be represented—which we can call $\mathbb{X}$ —is of dimensionality $3N^2$, with each dimension taking values between zero and the maximum measurable pixel intensity. The data samples in the support of p_{data} , however, constitute the manifold of the real data associated with some particular problem, typically occupying a very small part of the total space, $\mathbb{X}$. Similarly, the samples produced by the generator should also occupy only a small portion of $\mathbb{X}$.

Arjovsky et al. [26] showed that the support $p_g(\mathbf{x})$ and $p_{\text{data}}(\mathbf{x})$ lie in a lower-dimensional space than that corresponding to $\mathbb{X}$. The consequence of this is that $p_g(\mathbf{x})$ and $p_{\text{data}}(\mathbf{x})$ may have no overlap, and so there exists a nearly trivial discriminator that is capable of distinguishing real samples, $\mathbf{x} \sim p_{\text{data}}(\mathbf{x})$ from fake samples, $\mathbf{x} \sim p_g(\mathbf{x})$ with 100% accuracy. In this case, the discriminator error quickly converges to zero. Parameters of the generator may only be updated via the discriminator, so when this happens, the gradients used for updating parameters of the generator also converge to zero and may no longer be useful for updates to the generator. Arjovsky et al.’s explanations account for several of the symptoms related to GAN training [26].

Goodfellow et al. [1] also showed that when $\mathcal{D}$ is optimal, training $\mathcal{G}$ is equivalent to minimizing the Jensen–Shannon (JS) divergence between $p_g(\mathbf{x})$ and $p_{\text{data}}(\mathbf{x})$. If $\mathcal{D}$ is not optimal, the update may be less meaningful or inaccurate. This

theoretical insight has motivated research into cost functions based on alternative distances. Several of these are explored in the section “Alternative Formulations.”

Training tricks

One of the first major improvements in the training of GANs for generating images were the DCGAN architectures proposed by Radford et al. [5]. This work was the result of an extensive exploration of CNN architectures previously used in computer vision, and it resulted in a set of guidelines for constructing and training both the generator and discriminator. In the section “Convolutional GANs,” we alluded to the importance of strided and fractionally strided convolutions [27], which are key components of the architectural design. This allows both the generator and the discriminator to learn good upsampling and downsampling operations, which may contribute to improvements in the quality of image synthesis. More specifically to training, batch normalization [28] was recommended for use in both networks to stabilize training in deeper models.

Another suggestion was to minimize the number of fully connected layers used to increase the feasibility of training deeper models. Finally, Radford et al. [5] showed that using leaky rectifying linear units (ReLU) activation functions between the intermediate layers of the discriminator gave superior performance over using regular ReLUs.

Later, Salimans et al. [25] proposed further heuristic approaches for stabilizing the training of GANs. The first, feature matching, changes the objective of the generator slightly to increase the amount of information available. Specifically, the discriminator is still trained to distinguish between real and fake samples, but the generator is now trained to match the discriminator’s expected intermediate activations (features) of its fake samples with the expected intermediate activations of the real samples. The second, minibatch discrimination, adds an extra input to the discriminator, which is a feature that encodes the distance between a given sample in a minibatch and the other samples. This is intended to prevent mode collapse, as the discriminator can easily tell if the generator is producing the same outputs.

A third trick, heuristic averaging, penalizes the network parameters if they deviate from a running average of previous values, which can help convergence to an equilibrium. The fourth, virtual batch normalization, reduces the dependency of one sample on the other samples in the minibatch by calculating the batch statistics for normalization with the sample placed within a reference minibatch that is fixed at the beginning of training.

Finally, one-sided label smoothing makes the target for the discriminator 0.9 instead of one, smoothing the discriminator’s classification boundary, hence preventing an overly confident discriminator that would provide weak gradients for the generator. Sønderby et al. [29] advanced the idea of challenging

The representations that can be learned by GANs may be used in a variety of applications, including image synthesis, semantic image editing, style transfer, image superresolution, and classification.

the discriminator by adding noise to the samples before feeding them into the discriminator. Sønderby et al. [29] argued that one-sided label smoothing biases the optimal discriminator, while their technique, instance noise, moves the manifolds of the real and fake samples closer together, at the same time preventing the discriminator easily finding a discrimination boundary that completely separates the real and fake samples. In practice, this can be implemented by adding Gaussian noise to both the synthesized and real images, annealing the standard deviation over time. The same process was independently proposed by Arjovsky et al. [26].

Alternative formulations

The first part of this section considers other information-theoretic interpretations and generalizations of GANs. The second part looks at alternative cost functions that aim to directly address the problem of vanishing gradients.

Generalizations of the GAN cost function

Nowozin et al. [30] showed that GAN training may be generalized to minimize not only the JS divergence, but an estimate of f -divergences; these are referred to as f -GANs. The f -divergences include well-known divergence measures such as the KL-divergence. Nowozin et al. showed that the f -divergence may be approximated by applying the Fenchel conjugates of the desired f -divergence to samples drawn from the distribution of generated samples, after passing those samples through a discriminator [30]. They provide a list of Fenchel conjugates for commonly used f -divergences, as well as activation functions that may be used in the final layer of the generator network, depending on the choice of f -divergence. Having derived the generalized cost functions for training the generator and discriminator of an f -GAN, Nowozin et al. [30] observe that, in its raw form, maximizing the generator objective is likely to lead to weak gradients, especially at the start of training, and proposed an alternative cost function for updating the generator, which is less likely to saturate at the beginning of training. Nowozin et al. proposed that when the discriminator is trained, the derivative of the f -divergence on the ratio of the real and fake data distributions is estimated, while when the generator is trained only an estimate of the f -divergence is minimized. Uehara et al. [31] extend the f -GAN further, where in the discriminator step the ratio of the distributions of real and fake data are predicted, and in the generator step the f -divergence is directly minimized. Alternatives to the JS-divergence are also covered by Goodfellow [12].

Alternative cost functions to prevent vanishing gradients Arjovsky et al. [32] proposed the Wasserstein GAN (WGAN), a GAN with an alternative cost function that is derived from an approximation of the Wasserstein distance. Unlike the original GAN cost function, the WGAN is more likely to provide gradients that are useful for updating the generator. The cost

What sets GANs apart from these standard tools of signal processing is the level of complexity of the models that map vectors from latent space to image space.

function derived for the WGAN relies on the discriminator, which they refer to as the *critic*, being a k -Lipschitz continuous function; practically, this may be implemented by simply clipping the parameters of the discriminator. However, more recent research [33] suggested that weight clipping adversely reduces the capacity of the discriminator model, forcing it to learn simpler functions. Gulrajani et al. [33] proposed an

improved method for training the discriminator for a WGAN, by penalizing the norm of discriminator gradients with respect to data samples during training, rather than performing parameter clipping.

A brief comparison of GAN variants

GANs allow us to synthesize novel data samples from random noise, but they are considered difficult to train due partially to vanishing gradients. All GAN models that we have discussed in this article require careful hyperparameter tuning and model selection for training. However, perhaps the easier models to train are the adversarial autoencoder (AAE) and the WGAN. The AAE is relatively easy to train because the adversarial loss is applied to a fairly simple distribution in lower dimensions (than the image data). The WGAN [33], is designed to be easier to train, using a different formulation of the training objective that does not suffer from the vanishing gradient problem. The WGAN may also be trained successfully even without batch normalization; it is also less sensitive to the choice of nonlinearities used between convolutional layers.

Samples synthesized using a GAN or WGAN may belong to any class present in the training data. Conditional GANs provide an approach to synthesizing samples with user-specified content.

It is evident from various visualization techniques (Figure 6) that the organization of the latent space harbors some meaning, but vanilla GANs do not provide an inference model to allow data samples to be mapped to latent representations. Both BiGANs and ALI provide a mechanism to map image data to a latent space (inference), however, reconstruction quality suggests that they do not necessarily faithfully encode and decode samples. A very recent development shows that ALI may recover encoded data samples faithfully [21]. However, this model shares a lot in common with the AVB and AAE. These are autoencoders, similar to VAEs, where the latent space is regularized using adversarial training rather than a KL-divergence between encoded samples and a prior.

The structure of latent space

GANs build their own representations of the data they are trained on, and in doing so produce structured geometric vector spaces for different domains. This is a quality shared with other neural network models, including VAEs [23], as well as linguistic models such as word2vec [34]. In general, the domain of the data to be modeled is mapped to a vector space,

which has fewer dimensions than the data space, forcing the model to discover interesting structure in the data and represent it efficiently. This latent space is at the “originating” end of the generator network, and the data at this level of representation (the latent space) can be highly structured and may support high-level semantic operations [5]. Examples include the rotation of faces from trajectories through latent space, as well as image analogies that have the effect of adding visual attributes such as eyeglasses onto a “bare” face.

All (vanilla) GAN models have a generator that maps data from the latent space into the space to be modeled, but many GAN models have an encoder that additionally supports the inverse mapping [19], [20]. This becomes a powerful method for exploring and using the structured latent space of the GAN network. With an encoder, collections of labeled images can be mapped into latent spaces and analyzed to discover “concept vectors” that represent high-level attributes such as “smiling” or “wearing a hat.” These vectors can be applied at scaled offsets in latent space to influence the behavior of the generator (Figure 6). Similar to using an encoding process to model the distribution of latent samples, Gurumurthy et al. [35] propose modeling the latent space as a mixture of Gaussians and learning the mixture components that maximize the likelihood of generated data samples under the data generating distribution.

Applications of GANs

Discovering new applications for adversarial training of deep networks is an active area of research. We examine a few computer vision applications that have appeared in the literature and been subsequently refined. These applications were chosen to highlight some different approaches to using GAN-based representations for image manipulation, analysis, or characterization and do not fully reflect the potential breadth of application of GANs.

Using GANs for image classification places them within the broader context of machine learning and provides a useful quantitative assessment of the features extracted in unsuper-

GANs build their own representations of the data they are trained on, and in doing so produce structured geometric vector spaces for different domains.

vised learning. Image synthesis remains a core GAN capability and is especially useful when the generated image can be subject to pre-existing constraints. Superresolution [36]–[38] offers an example of how an existing approach can be supplemented with an adversarial loss component to achieve higher-quality results. Finally, image-to-image translation demonstrates how GANs offer a general-purpose solution to a family of tasks

that require automatically converting an input image into an output image.

Classification and regression

After GAN training is complete, the neural network can be reused for other downstream tasks. For example, outputs of the convolutional layers of the discriminator can be used as a feature extractor, with simple linear models fitted on top of these features using a modest quantity of (image, label) pairs [5], [25]. The quality of the unsupervised representations within a DCGAN network have been assessed by applying a regularized L2-SVM classifier to a feature vector extracted from the (trained) discriminator [5]. Good classification scores were achieved using this approach on both supervised and semisupervised data sets, even those that were disjoint from the original training data.

The quality of the data representation may be improved when adversarial training includes jointly learning an inference mechanism such as with ALI [19]. A representation vector was built using last three hidden layers of the ALI encoder, a similar L2-SVM classifier, yet achieved a misclassification rate significantly lower than the DCGAN [19]. Additionally, ALI has achieved state-of-the-art classification results when label information is incorporated into the training routine.

When labeled training data is in limited supply, adversarial training may also be used to synthesize more training samples. Shrivastava et al. [39] use GANs to refine synthetic images while maintaining their annotation information. By training models only on GAN-refined synthetic images (i.e., no real training data) Shrivastava et al. [39] achieved state-of-the-art performance on pose- and gaze-estimation tasks. Similarly, good results were obtained for gaze estimation and prediction



FIGURE 6. An example of applying a “smile vector” with an ALI model [19]. The first image is an example of an unsmiling woman and the last is an example of a woman smiling. A $\mathbf{z}$ value for the first image is inferred, $\mathbf{z}_1$ and for the last, $\mathbf{z}_2$. Interpolating along a vector that connects $\mathbf{z}_1$ and $\mathbf{z}_2$, gives $\mathbf{z}$ values that may be passed through a generator to synthesize novel samples. Note the implication: a displacement vector in latent space traverses smile “intensity” in image space. (Figure used courtesy of Tom White.)

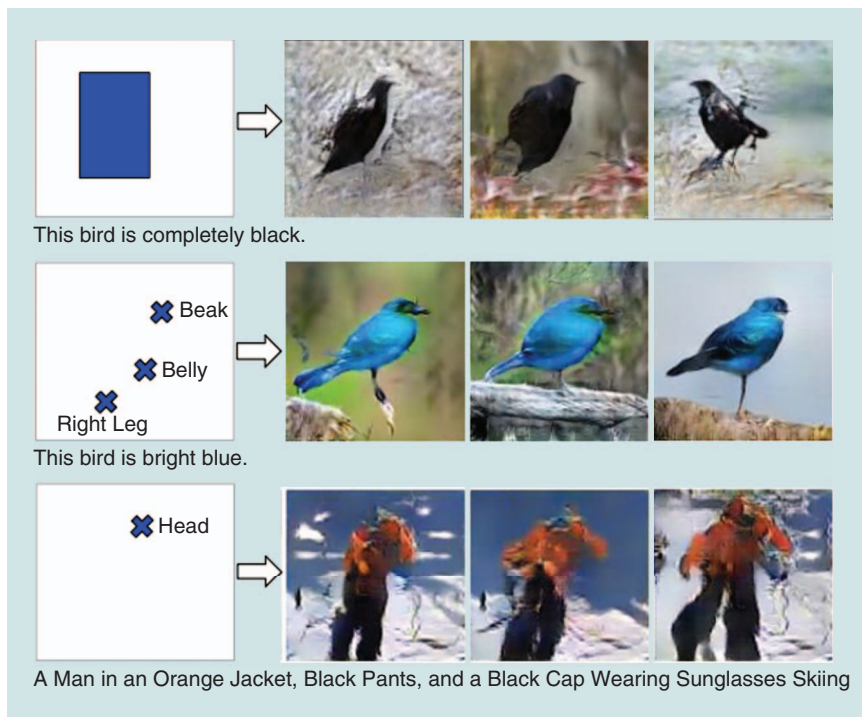


FIGURE 7. Examples of image synthesis using the GAWWN. In the GAWWN, images are conditioned on both text descriptions and image location specified as either a keypoint or bounding box. (Figure reproduced from [44] with permission.)

using a spatiotemporal GAN architecture [40]. In some cases, models trained on synthetic data do not generalize well when applied to real data [3]. Bousmalis et al. [3] propose to address this problem by adapting synthetic samples from a source domain to match a target domain using adversarial training. Additionally, Liu et al. [41] propose using multiple GANs—one per domain—with tied weights to synthesize pairs of corresponding images samples from different domains. Because the quality of generated samples is hard to quantitatively judge across models, classification tasks are likely to remain an important quantitative tool for performance assessment of GANs, even as new and diverse applications in computer vision are explored.

Image synthesis

Much of the recent GAN research focuses on improving the quality and utility of the image-generation capabilities. The LAPGAN model introduced a cascade of convolutional networks within a Laplacian pyramid framework to generate images in a coarse-to-fine fashion [13]. A similar approach is used by Huang et al. [42] with GANs operating on intermediate representations rather than lower-resolution images.

LAPGAN also extended the conditional version of the GAN model where both $\mathcal{G}$ and $\mathcal{D}$ networks receive additional label information as input; this technique has proved useful and is now a common practice to improve image quality. This idea of GAN conditioning was later extended to incorporate natural language. For example, Reed et al. [43] used a GAN architecture to synthesize images from text descriptions, which one

might describe as *reverse captioning*. For example, given a text caption of a bird such as “white with some black on its head and wings and a long, orange beak,” the trained GAN can generate several plausible images that match the description.

In addition to conditioning on text descriptions, the generative adversarial what-where network (GAWWN) conditions on image location [44]. The GAWWN system supported an interactive interface in which large images could be built up incrementally with textual descriptions of parts and user-supplied bounding boxes (Figure 7).

Conditional GANs not only allow us to synthesize novel samples with specific attributes, they also allow us to develop tools for intuitively editing images; e.g., changing the hairstyle of a person in an image, making them wear glasses, or editing the image so they appear younger [35]. Additional applications of GANs to image editing include work by Zhu and Brock et al. [2], [45].

Image-to-image translation

Conditional adversarial networks are well suited for translating an input image into an output image, which is a recurring theme in computer graphics, image processing, and computer vision. The pix2pix model offers a general-purpose solution to this family of problems [46]. In addition to learning the mapping from input image to output image, the pix2pix model also constructs a loss function to train this mapping. This model has demonstrated effective results for different problems of computer vision that had previously required separate machinery, including semantic segmentation, generating maps from aerial photos, and colorization of black and white images. Wang et al. present a similar idea, using GANs to first synthesize surface-normal maps (similar to depth maps) and then map these images to natural scenes.

CycleGAN [4] extends this work by introducing a cycle consistency loss that attempts to preserve the original image after a cycle of translation and reverse translation. In this formulation, matching pairs of images are no longer needed for training. This makes data preparation much simpler, and opens the technique to a larger family of applications. For example, artistic style transfer [47] renders natural images in the style of artists, such as Picasso or Monet, by simply being trained on an unpaired collection of paintings and natural images (Figure 8).

Superresolution

Superresolution allows a high-resolution image to be generated from a lower-resolution image, with the trained model inferring photo-realistic details while upsampling. The SRGAN

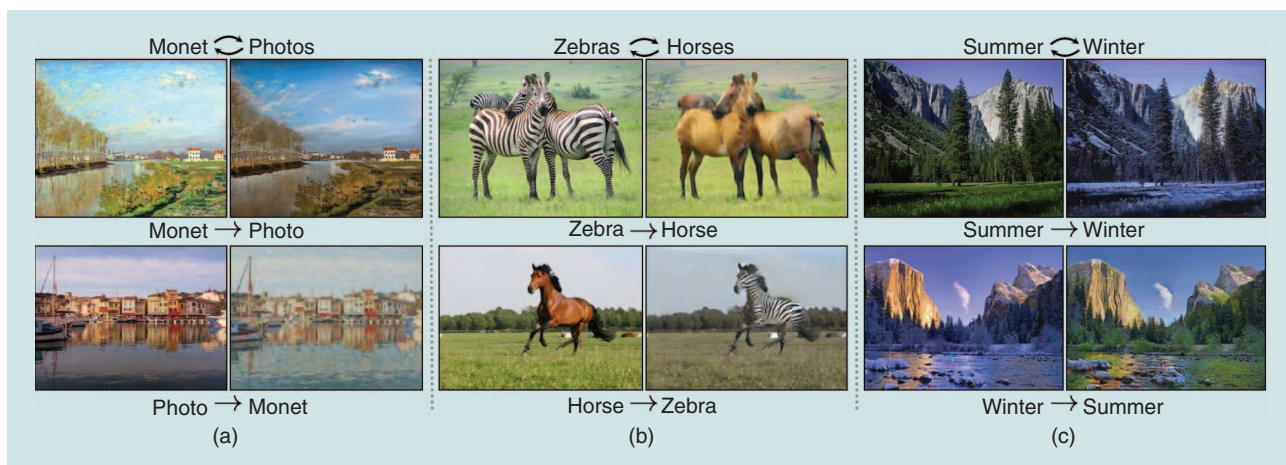


FIGURE 8. The CycleGAN model learns image to image translations between two unordered image collections. Shown here are the examples of bidirectional image mappings: (a) Monet paintings to landscape photos, (b) zebras to horses, and (c) summer to winter photos in Yosemite National Park. (Figure reproduced from [4] with permission.)

model [36] extends earlier efforts by adding an adversarial loss component, which constrains images to reside on the manifold of natural images.

The SRGAN generator is conditioned on a low-resolution image and infers photo-realistic natural images with $4 \times$ upscaling factors. Unlike most GAN applications, the adversarial loss is one component of a larger loss function, which also includes perceptual loss from a pretrained classifier, and a regularization loss that encourages spatially coherent images. In this context, the adversarial loss constrains the overall solution to the manifold of natural images, producing perceptually more convincing solutions.

Customizing deep-learning applications can often be hampered by the availability of relevant curated training data sets. However, SRGAN is straightforward in customizing to specific domains, as new training image pairs can easily be constructed by downsampling a corpus of high-resolution images. This is an important consideration in practice, since the inferred photo-realistic details that the GAN generates will vary depending on the domain of images used in the training set.

Discussion

Open questions

GANs have attracted considerable attention due to their ability to leverage vast amounts of unlabeled data. While much progress has been made to alleviate some of the challenges related to training and evaluating GANs, there still remain several open challenges.

Mode collapse

As articulated in the section “Training GANs,” a common problem of GANs involves the generator collapsing to produce a small family of similar samples (partial collapse) and, in the worst case, producing simply a single sample (complete collapse) [26], [48]. Diversity in the generator can be increased by practical hacks to balance the distribution of samples produced

by the discriminator for real and fake batches, or by employing multiple GANs to cover the different modes of the probability distribution [49]. Yet another solution to alleviate mode collapse is to alter the distance measure used to compare statistical distributions. Arjovsky [32] proposed to compare distributions based on a Wasserstein distance rather than a KL-based divergence (DCGAN [5]) or a total-variation distance (energy-based GAN [50]). Metz et al. [51] proposed unrolling the discriminator for several steps, i.e., letting it calculate its updates on the current generator for several steps, and then using the “unrolled” discriminators to update the generator using the normal minimax objective. As normal, the discriminator only trains on its update from one step, but the generator now has access to how the discriminator would update itself. With the usual one step generator objective, the discriminator will simply assign a low probability to the generator’s previous outputs, forcing the generator to move, resulting either in convergence, or an endless cycle of mode hopping. However, with the unrolled objective, the generator can prevent the discriminator from focusing on the previous update, and update its own generations with the foresight of how the discriminator would have responded.

Training instability—saddle points

In a GAN, the Hessian of the loss function becomes indefinite. The optimal solution, therefore, lies in finding a saddle point rather than a local minimum. In deep learning, a large number of optimizers depend only on the first derivative of the loss function; converging to a saddle point for GANs requires good initialization. By invoking the stable manifold theorem from nonlinear systems theory, Lee et al. [52] showed that, were we to select the initial points of an optimizer at random, gradient descent would not converge to a saddle with probability one (also see [25] and [53]). Additionally, Mescheder et al. [54] have argued that convergence of a GAN’s objective function suffers from the presence of a zero real part of the Jacobian matrix as well as eigenvalues with large imaginary parts. This is disheartening for GAN training; yet, due to the existence of second-order

optimizers, not all hope is lost. Unfortunately, Newton-type methods have compute-time complexity that scales cubically or quadratically with the dimension of the parameters. Therefore, another line of questions lies in applying and scaling second-order optimizers for adversarial training.

A more fundamental problem is the existence of an equilibrium for a GAN. Using results from Bayesian nonparametrics, Arora et al. [48] connects the existence of the equilibrium to a finite mixture of neural networks—this means that, below a certain capacity, no equilibrium might exist. On a closely related note, it has also been argued that, while GAN training can appear to have converged, the trained distribution could still be far away from the target distribution. To alleviate this issue, Arora et al. [48] propose a new measure called the *neural net distance*.

Evaluating generative models

How can one gauge the fidelity of samples synthesized by a generative models? Should we use a likelihood estimation? Can a GAN trained using one methodology be compared to another (model comparison)? These are open-ended questions that are not only relevant for GANs but also for probabilistic models, in general. Theis [55] argued that evaluating GANs using different measures can lead conflicting conclusions about the quality of synthesized samples; the decision to select one measure over another depends on the application.

Conclusions

The explosion of interest in GANs is driven not only by their potential to learn deep, highly nonlinear mappings from a latent space into a data space and back but also by their potential to make use of the vast quantities of unlabeled image data that remain closed to deep representation learning. Within the subtleties of GAN training, there are many opportunities for developments in theory and algorithms, and with the power of deep networks, there are vast opportunities for new applications.

Acknowledgments

We would like to thank David Warde-Farley for his valuable feedback on previous revisions of the article. Antonia Creswell acknowledges the support of the Engineering and Physical Sciences Research Council through a doctoral training scholarship.

Authors

Antonia Creswell (ac2211@ic.ac.uk) received her first-class degree from Imperial College London in biomedical engineering in 2011 and is currently a Ph.D. degree student in the Biologically Inspired Computer Vision Group at Imperial College London. The focus of her Ph.D. research is improving the training of generative adversarial networks and applying them to visual search and learning representations in unlabeled sources of image data.

Tom White (tom@sixdozen.com) received his B.S. degree in mathematics from the University of Georgia and his M.S. degree in media arts and sciences from the Massachusetts Institute of Technology. He is currently a senior lecturer in the School of Design at Victoria University of Wellington, New Zealand. His current research focuses on exploring the growing use of con-

structive machine learning in computational design and the creative potential of human designers working collaboratively with artificial neural networks during the exploration of design ideas and prototyping.

Vincent Dumoulin (vi.dumoulin@gmail.com) received his B.Sc. degree in physics and computer science from the University of Montréal, Canada. He is a doctoral candidate at the Montréal Institute for Learning Algorithms under the cosupervision of Yoshua Bengio and Aaron Courville, working on deep-learning approaches to generative modeling.

Kai Arulkumaran (kailash.arulkumaran13@imperial.ac.uk) received his B.A. degree in computer science from the University of Cambridge, United Kingdom, in 2012 and his M.Sc. degree in biomedical engineering from Imperial College London in 2014, where he is currently a Ph.D. candidate in the Department of Bioengineering. He was a research intern at Twitter Magic Pony and Microsoft Research in 2017. His research focus is deep reinforcement learning and computer vision for visuomotor control.

Biswa Sengupta (biswasengupta@gmail.com) received his B.Eng. (honors) degree in electrical and computer engineering in 2004 and his M.Sc. degree in theoretical computer science in 2005 from the University of York, United Kingdom. He received his second M.Sc. degree in neural and behavioral sciences in 2007 from the Max Planck Institute for Biological Cybernetics, Germany, and his Ph.D. degree in theoretical neuroscience in 2011 from the University of Cambridge, United Kingdom. He received further training in Bayesian statistics and differential geometry at the University College London and University of Cambridge before leading Cortextica Vision Systems as its chief scientist. Currently, he is a visiting scientist at Imperial College London, and he is also leading machine-learning research at Noah's Ark Lab of Huawei Technologies United Kingdom.

Anil A. Bharath (a.bharath@imperial.ac.uk) received his B. Eng. degree in electronic and electrical engineering from University College London in 1988, and a Ph.D. degree in signal processing from Imperial College London in 1993, where he is currently a reader in the Department of Bioengineering. He is an academic fellow of Imperial's Data Science Institute and a fellow of the Institution of Engineering and Technology. He was an academic visitor in the Signal Processing Group at the University of Cambridge in 2006. He is a cofounder of Cortextica Vision Systems. His research interest is in deep architectures for visual inference.

References

- [1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Proc. Advances Neural Information Processing Systems Conf.*, 2014, pp. 2672–2680.
- [2] J.-Y. Zhu, P. Krähenbühl, E. Shechtman, and A. A. Efros, "Generative visual manipulation on the natural image manifold," in *Proc. European Conf. Computer Vision*, 2016, pp. 597–613.
- [3] K. Bousmalis, N. Silberman, D. Dohan, D. Erhan, and D. Krishnan, "Unsupervised pixel-level domain adaptation with generative adversarial networks," in *Proc. IEEE Conf. Computer Vision Pattern Recognition*, 2016, pp. 3722–3731.
- [4] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, (2017). Unpaired image-to-image translation using cycle-consistent adversarial networks, *Proc. Int. Conf. Computer Vision*. [Online]. Available: <https://arxiv.org/abs/1703.10593>

- [5] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *Proc. 5th Int. Conf. Learning Representations Workshop Track*, 2016.
- [6] A. Creswell and A. A. Bharath, "Adversarial training for sketch retrieval," in *Proc. European Conf. Computer Vision Workshops*, Amsterdam, The Netherlands, 2016, pp. 798–809.
- [7] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [8] H. Hotelling, "Analysis of a complex of statistical variables into principal components," *J. Educ. Psychol.*, vol. 24, no. 6, pp. 417, 1933.
- [9] I. J. Goodfellow, "On distinguishability criteria for estimating generative models," in *Proc. Int. Conf. Learning Representations Workshop Track*, 2015.
- [10] M. Gutmann and A. Hyvärinen, "Noise-contrastive estimation: A new estimation principle for unnormalized statistical models," *Artif. Intell. Statist.*, vol. 1, no. 2, p. 6, 2010.
- [11] Y. Bengio, L. Yao, G. Alain, and P. Vincent, "Generalized denoising autoencoders as generative models," in *Proc. Advances Neural Information Processing Systems Conf.*, 2013, pp. 899–907.
- [12] I. Goodfellow. (2016). NIPS 2016 tutorial: Generative adversarial networks, *Proc. Neural Information Processing Systems Conf.* [Online]. Available: <https://arxiv.org/abs/1701.00160>
- [13] E. L. Denton, S. Chintala, A. Szlam, and R. Fergus, "Deep generative image models using a Laplacian pyramid of adversarial networks," in *Proc. Advances Neural Information Processing Systems Conf.*, 2015, pp. 1486–1494.
- [14] J. Wu, C. Zhang, T. Xue, B. Freeman, and J. Tenenbaum, "Learning a probabilistic latent space of object shapes via 3D generative-adversarial modeling," in *Proc. Advances Neural Information Processing Systems Conf.*, 2016, pp. 82–90.
- [15] M. Mirza and S. Osindero, "Conditional generative adversarial nets," *arXiv Preprint*, arXiv:1411.1784, 2014.
- [16] X. Chen, Y. Duan, R. Houthoofd, J. Schulman, I. Sutskever, and P. Abbeel, "Infogan: Interpretable representation learning by information maximizing generative adversarial nets," in *Proc. Advances Neural Information Processing Systems Conf.*, 2016, pp. 2172–2180.
- [17] A. Creswell and A. A. Bharath, "Inverting the generator of a generative adversarial network," in *Proc. Neural Information Processing Systems Workshop Adversarial Training*, 2016.
- [18] Z. C. Lipton and S. Tripathi, "Precise recovery of latent vectors from generative adversarial networks," in *Proc. Int. Conf. Learning Representations Workshop Track*, 2017.
- [19] V. Dumoulin, I. Belghazi, B. Poole, O. Mastropietro, A. Lamb, M. Arjovsky, and A. Courville, "Adversarially learned inference," in *Proc. Int. Conf. Learning Representations*, 2017.
- [20] J. Donahue, P. Krähenbühl, and T. Darrell, "Adversarial feature learning," in *Proc. Int. Conf. Learning Representations*, 2017.
- [21] C. Li, H. Liu, C. Chen, Y. Pu, L. Chen, R. Henao, and L. Carin, "Towards understanding adversarial learning for joint distribution matching," in *Proc. Advances Neural Information Processing Systems Conf.*, 2017.
- [22] A. Makhzani, J. Shlens, N. Jaitly, and I. Goodfellow. (2016). Adversarial autoencoders, *Proc. Int. Conf. Learning Representations*. [Online]. Available: <http://arxiv.org/abs/1511.05644>
- [23] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," in *Proc. 2nd Int. Conf. Learning Representations*, 2014.
- [24] L. M. Mescheder, S. Nowozin, and A. Geiger. (2017). Adversarial variational Bayes: Unifying variational autoencoders and generative adversarial networks. [Online]. Available: <http://arxiv.org/abs/1701.04722>
- [25] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved techniques for training GANS," in *Proc. Advances Neural Information Processing Systems Conf.*, 2016, pp. 2226–2234.
- [26] M. Arjovsky and L. Bottou, "Towards principled methods for training generative adversarial networks," in *Proc. Neural Information Processing Systems Conf. Workshop Adversarial Training*, 2016.
- [27] E. Shelhamer, J. Long, and T. Darrell, "Fully convolutional networks for semantic segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 4, pp. 640–651, 2017.
- [28] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. 32nd Int. Conf. Machine Learning*, 2015, pp. 448–456.
- [29] C. K. Sønderby, J. Caballero, L. Theis, W. Shi, and F. Huszár, "Amortised map inference for image super-resolution," in *Proc. Int. Conf. Learning Representations*, 2017.
- [30] S. Nowozin, B. Cseke, and R. Tomioka, "F-GAN: Training generative neural samplers using variational divergence minimization," in *Proc. Advances Neural Information Processing Systems Conf.*, 2016, pp. 271–279.
- [31] M. Uehara, I. Sato, M. Suzuki, K. Nakayama, and Y. Matsuo, "Generative adversarial nets from a density ratio estimation perspective," *arXiv Preprint*, arXiv:1610.02920, 2016.
- [32] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein GAN," in *Proc. 34th Int. Conf. Machine Learning*, 2017, pp. 214–223.
- [33] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville, "Improved training of wasserstein GANS," in *Proc. Advances Neural Information Processing Systems Conf.*, 2017.
- [34] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *Proc. Int. Conf. Learning Representations*, 2013.
- [35] S. Gurumurthy, R. K. Sarvadevabhatla, and V. B. Radhakrishnan, "Deligan: Generative adversarial networks for diverse and limited data," in *Proc. IEEE Conf. Computer Vision Pattern Recognition*, 2017, pp. 166–174.
- [36] C. Ledig, L. Theis, F. Huszár, J. Caballero, A. Aitken, A. Tejani, J. Totz, Z. Wang, and W. Shi, "Photo-realistic single image super-resolution using a generative adversarial network," in *Proc. IEEE Conf. Computer Vision Pattern Recognition*, 2017, pp. 4681–4690.
- [37] X. Yu and F. Porikli, "Ultra-resolving face images by discriminative generative networks," in *Proc. European Conf. Computer Vision*, 2016, pp. 318–333.
- [38] X. Yu and F. Porikli, "Hallucinating very low-resolution unaligned and noisy face images by transformative discriminative autoencoders," in *Proc. IEEE Conf. Computer Vision Pattern Recognition*, 2017, pp. 3760–3768.
- [39] A. Shrivastava, T. Pfister, O. Tuzel, J. Susskind, W. Wang, and R. Webb, "Learning from simulated and unsupervised images through adversarial training," in *Proc. IEEE Conf. Computer Vision Pattern Recognition*, 2016, pp. 2107–2116.
- [40] M. Zhang, K. T. Ma, J. H. Lim, Q. Zhao, and J. Feng, "Deep future gaze: Gaze anticipation on egocentric videos using adversarial networks," in *Proc. IEEE Conf. Computer Vision Pattern Recognition*, 2017, pp. 4372–4381.
- [41] M.-Y. Liu and O. Tuzel, "Coupled generative adversarial networks," in *Proc. Advances Neural Information Processing Systems Conf.*, 2016, pp. 469–477.
- [42] X. Huang, Y. Li, O. Poursaeed, J. Hopcroft, and S. Belongie, "Stacked generative adversarial networks," in *Proc. IEEE Conf. Computer Vision Pattern Recognition*, 2016.
- [43] S. Reed, Z. Akata, X. Yan, L. Logeswaran, B. Schiele, and H. Lee. (2016). Generative adversarial text to image synthesis, *Proc. Int. Conf. Machine Learning*. [Online]. Available: <https://arxiv.org/abs/1605.05396>
- [44] S. E. Reed, Z. Akata, S. Mohan, S. Tenka, B. Schiele, and H. Lee, "Learning what and where to draw," in *Proc. Advances Neural Information Processing Systems Conf.*, 2016, pp. 217–225.
- [45] A. Brock, T. Lim, J. M. Ritchie, and N. Weston, "Neural photo editing with introspective adversarial networks," in *Proc. 6th Int. Conf. Learning Representations*, 2017.
- [46] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. IEEE Conf. Computer Vision Pattern Recognition*, 2016, pp. 1125–1134.
- [47] C. Li and M. Wand, "Precomputed real-time texture synthesis with Markovian generative adversarial networks," in *Proc. European Conf. Computer Vision*, 2016, pp. 702–716.
- [48] S. Arora, R. Ge, Y. Liang, T. Ma, and Y. Zhang, "Generalization and equilibrium in generative adversarial nets (GANs)," in *Proc. 34th Int. Conf. Machine Learning*, 2017, pp. 224–232.
- [49] I. Tolstikhin, S. Gelly, O. Bousquet, C.-J. Simon-Gabriel, and B. Schölkopf, "ADAGAN: Boosting generative models," *arXiv Preprint* arXiv:1701.02386. [Online]. Available: <https://arxiv.org/abs/1701.02386>
- [50] J. Zhao, M. Mathieu, and Y. LeCun. (2017). Energy-based generative adversarial network, *Proc. Int. Conf. Learning Representations*. [Online]. Available: <https://arxiv.org/abs/1609.03126>
- [51] L. Metz, B. Poole, D. Pfau, and J. Sohl-Dickstein. (2017). Unrolled generative adversarial networks, *Proc. Int. Conf. Learning Representations*. [Online]. Available: <https://arxiv.org/abs/1611.02163>
- [52] J. D. Lee, M. Simchowitz, M. I. Jordan, and B. Recht, "Gradient descent only converges to minimizers," in *Proc. Conf. Learning Theory*, 2016, pp. 1246–1257.
- [53] R. Pemantle, "Nonconvergence to unstable points in urn models and stochastic approximations," *Ann. Probab.*, vol. 18, no. 2, pp. 698–712, Apr. 1990.
- [54] L. M. Mescheder, S. Nowozin, and A. Geiger. (2017). The numerics of GANS, *Proc. Advances Neural Information Processing Systems Conf.* [Online]. Available: <http://arxiv.org/abs/1705.10461>
- [55] L. Theis, A. van den Oord, and M. Bethge, "A note on the evaluation of generative models," in *Proc. Int. Conf. Learning Representations*.